

Identifying Pseudo Random Number Generators

From Statistical Test Output

Christoph Hagenauer, Jarrett Kizer

Department, University of South Carolina Beaufort, Bluffton, SC 29909

Introduction

Description of project

Random numbers are used in various computational programs, such as cryptography, modeling, lottery, and many more. In the first step of our research, we want to see if it is possible to predict the type of generator based on the output of TestU01's big crush battery. This battery consists of more than 106 statistical tests which help evaluate the randomness of the generator. For this purpose, we used six generators from the SPRNG library with various seeds and parameters. In the end, we collected close to 6000 rows of training data. With this data, we attempted to use machine learning to predict the generator that created a specific set of output from the big crush battery. We were able to build a model which achieves an accuracy of 79.7% after cross-validation. This accuracy reaches 99.9% if we predict the family of generators instead of the specific generators. In the future, we hope to include more generators and attempt to determine their input parameters based on the TestU01 big crush battery.

Hypothesis

- Null Hypothesis: It is not possible to identify a PRNG based on the results of statistical tests.
- Alternate Hypothesis: It is possible to identify a PRNG based on the results of statistical tests.

Methods

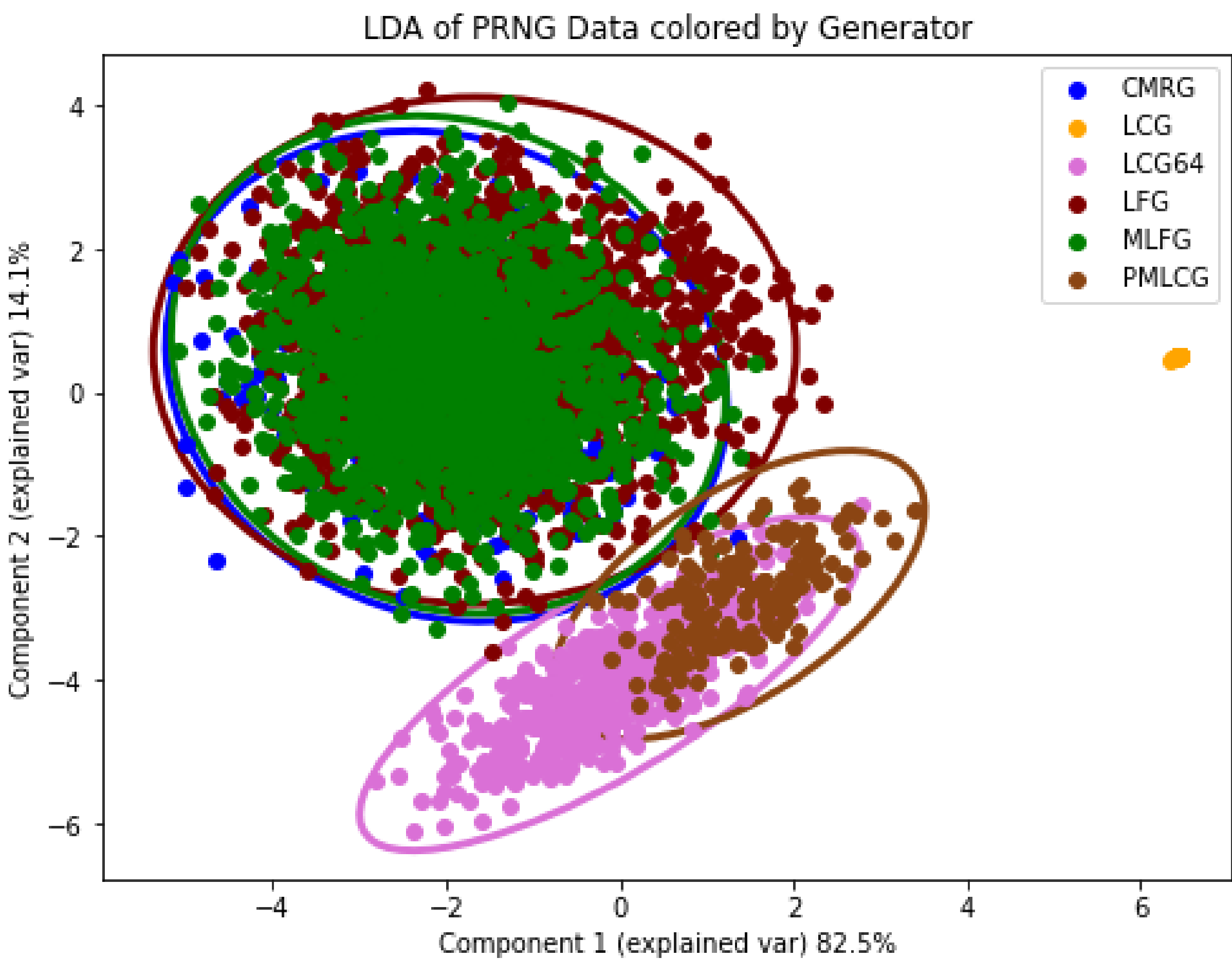
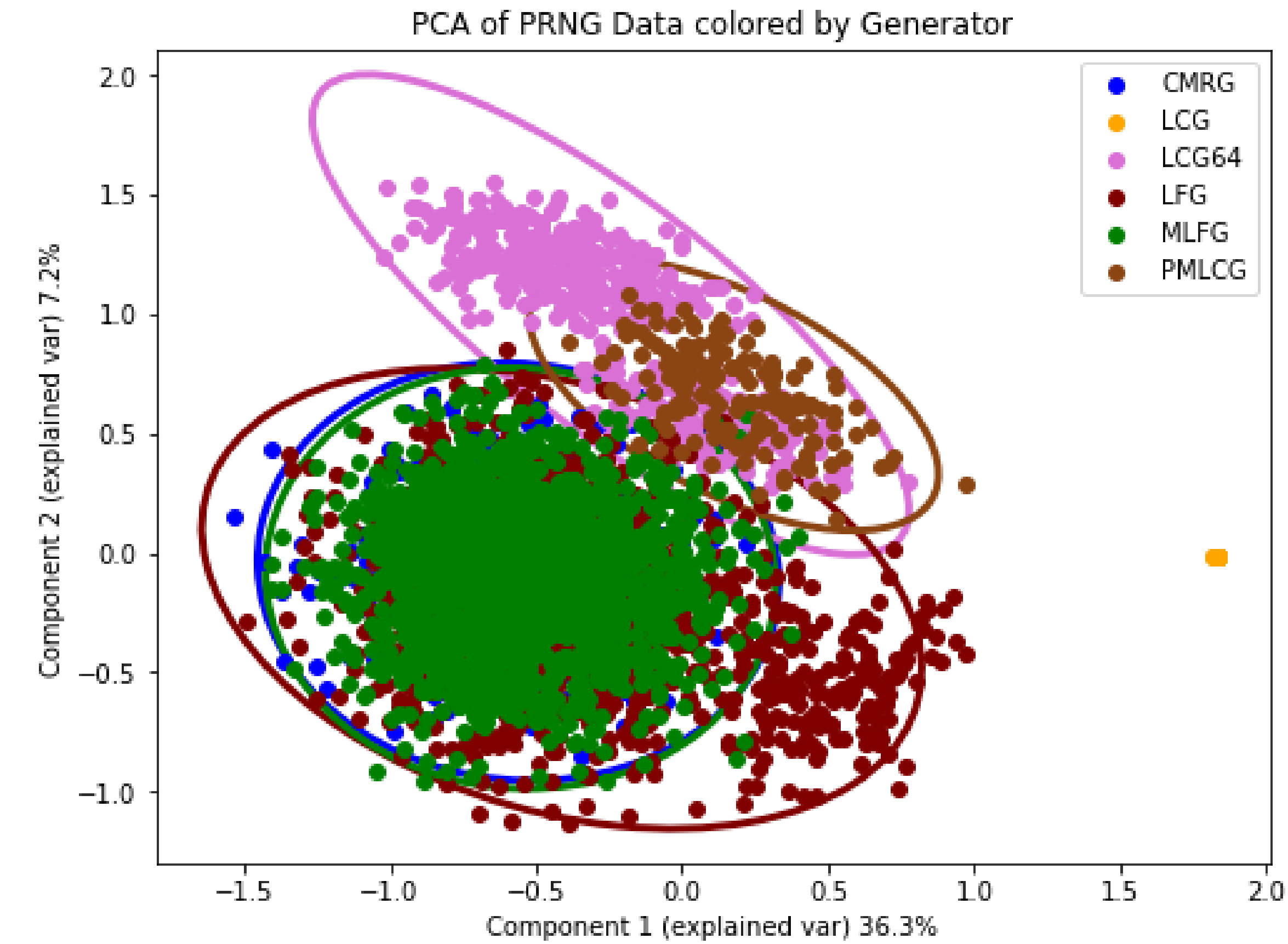
- Details:
- Results from 6 PRNG's of the SPRNG library through the TestU01 big crush generator
 - This dataset is run through various models and the results are cataloged

Procedure:

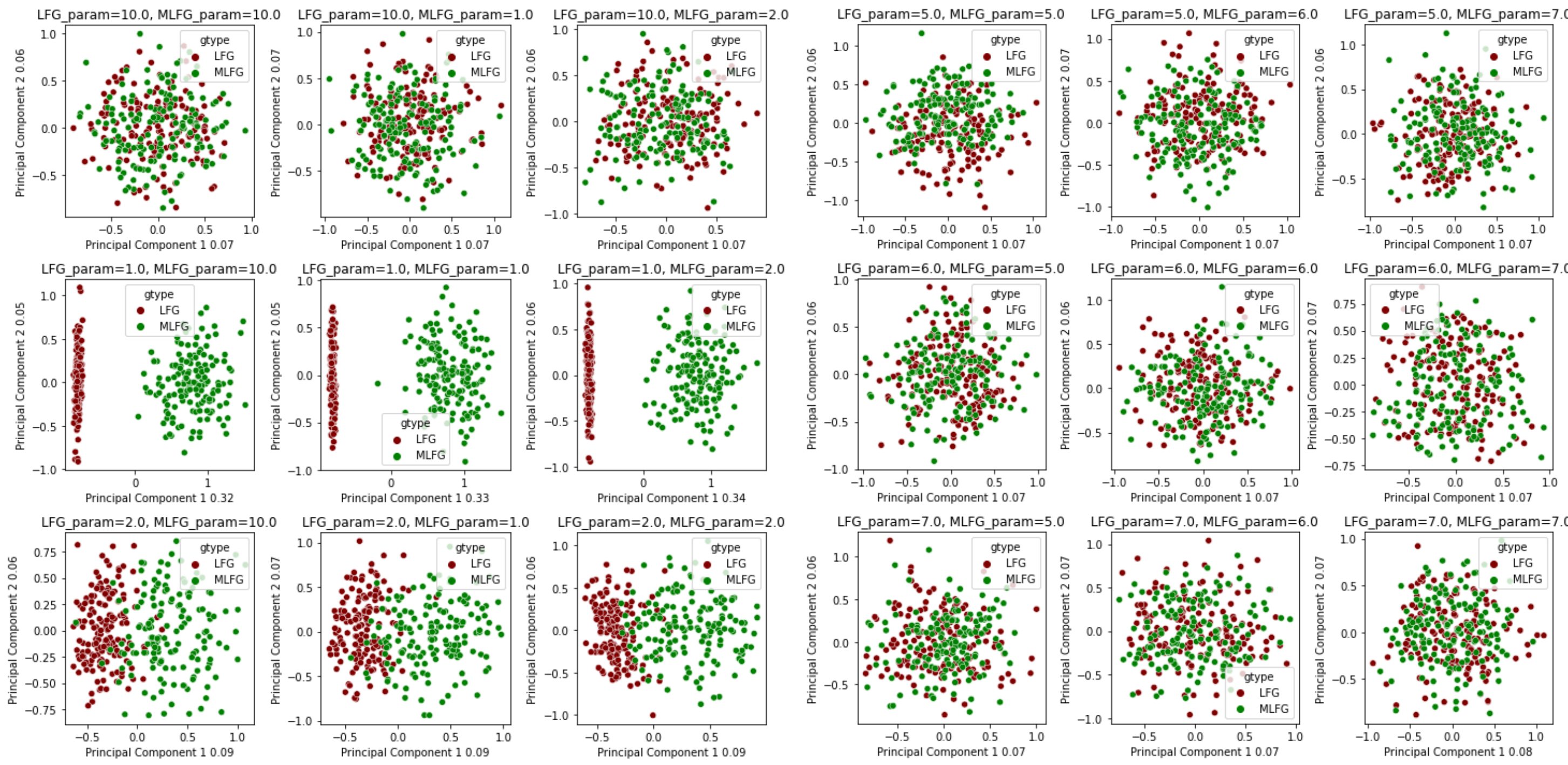
- First, we set up the tests to run on USCB's High Performance Computing server. A single test took 3-5 hours.
- We combined all our results into a single csv file.
- We used a chi squared test to select the 23 most influential features.
- We ran various analyses of the combined data file. So that we could get an understanding of the data.
- Finally, we ran various models and measured the scores to determine the best fit.
- We also used these models to determine what the most important features were according to the model.

Table 1. Summary of the Statistical Test Outputs

Generator Type	Parameters Tested	avg Pass Rate (a=0.05)	Total tested	% of Total
CMRG	3	90%	477	8.3%
LCG	7	52.8%	1113	19.4%
LCG64	3	82.9%	477	8.3%
LFG	11	87.2%	1749	30.6%
MLFG	11	89.8%	1749	30.6%
PMLCG	1	83.7%	159	2.8%



Graph 3. Showing PCA separations of MLFG and LFG subsetting by Parameters



Here we showcase a portion of the graphs where we attempted to separate the subsets of MLFG and LFG by the parameters used during their initialization. This is a 6 by 3 graphs, the full version is 11 by 11. We found 4 subsets from LFG to be distinct from all MLFG subsets.

Confusion Matrix of Model Predictions (test set) Normalized by Row

	CMRG	LCG	LCG64	LFG	MLFG	PMLCG
True Labels	CMRG	1.000	0.000	0.000	0.000	0.000
LCG	0.000	1.000	0.000	0.000	0.000	0.000
LCG64	0.000	0.000	1.000	0.000	0.000	0.000
LFG	0.007	0.000	0.000	0.416	0.578	0.000
MLFG	0.000	0.000	0.000	0.053	0.947	0.000
PMLCG	0.000	0.000	0.000	0.000	0.000	1.000

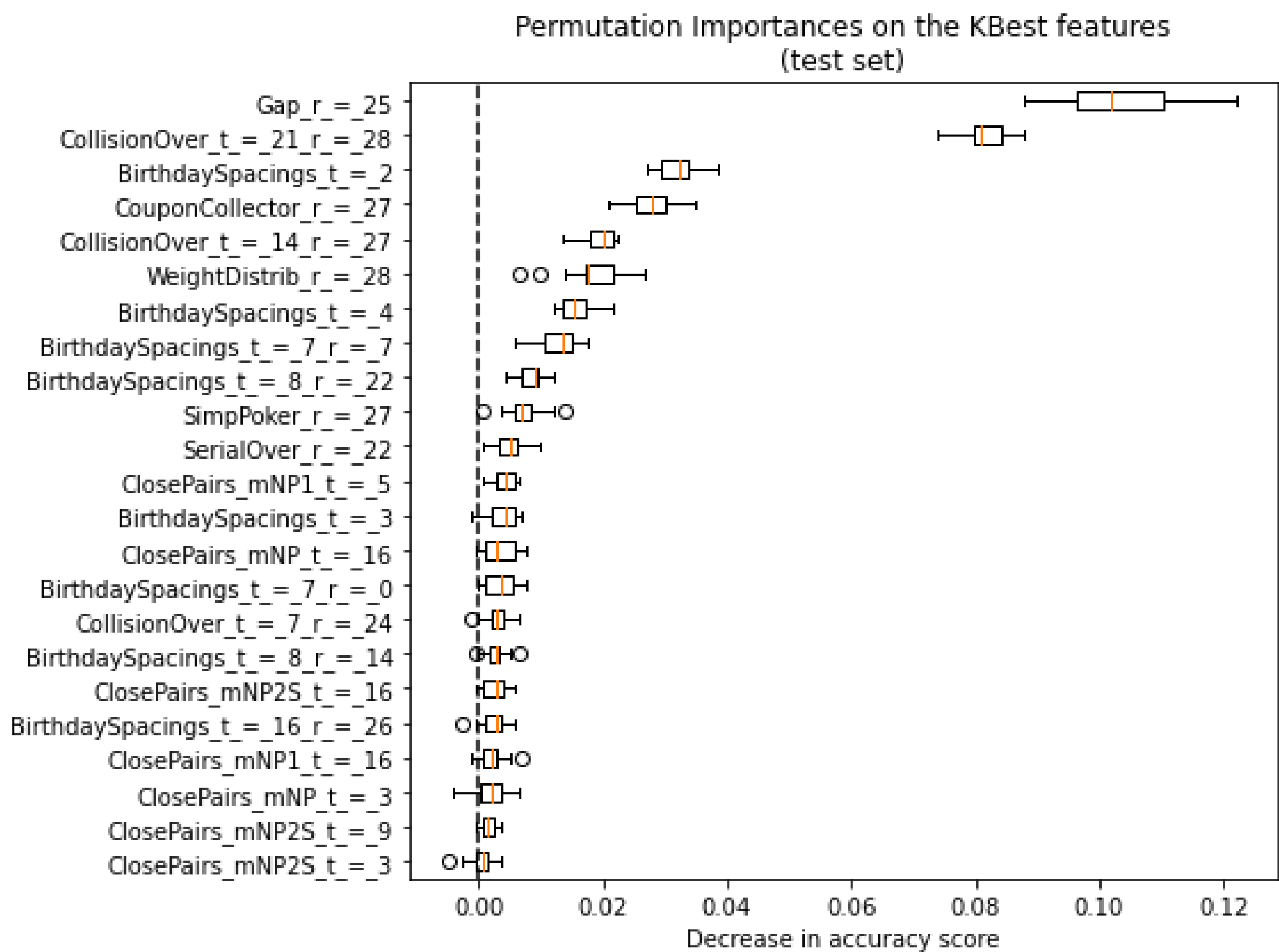


Table 2. Model Performance Metrics (Test Set)

Generator Type	precision	recall	f1-score	support
CMRG	0.98	1	0.99	119
LCG	1	1	1	278
LCG64	1	1	1	119
LFG	0.89	0.42	0.57	438
MLFG	0.62	0.95	0.75	437
PMLCG	1	1	1	40
accuracy			0.81	1431
macro avg	0.91	0.89	0.88	1431
weighted avg	0.85	0.81	0.79	1431

Discussion & Implications

- We can separate the 6 generators from the SPRNG library based on their TestU01 results.
- We may be able to identify more generators from different libraries.
- MLFG and LFG are from the same family of generators and are difficult to identify. If we continue to test generators from the same families, will they also be difficult to identify.
- In the future we want to try to predict the generator and the individual parameters for that generator.

These are promising initial results, but we will continue to dig deeper by generating data for more PRNGs. We hope to see similar separation amongst other generators.

References

- SPRNG version 5.0 M. Mascagni and A. Srinivasan (2000), "Algorithm 806: SPRNG: A Scalable Library for Pseudorandom Number Generation," ACM Transactions on Mathematical Software, 26: 436-461
- P. L'Ecuyer and R. Simard, TestU01: A C Library for Empirical Testing of Random Number Generators ACM Transactions on Mathematical Software, Vol. 33, article 22, 2007.
- Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.